GISMA University of Applied Sciences

M605 Advanced Databases

Design and Implementation of a Normalized SQL-Based E-Commerce Database System

Student Name	Arun Singh Chauhan
Student ID	GH1052389
Module	M605 Advanced Databases
Assignment	Design and Implementation of a Normalized SQL-Based E-Commerce Database System
Semester	Winter 2026

Project Resources

GitHub Repository	https://github.com/yourusername/your-repo
YouTube Demo	https://youtube.com/watch?v=your-video-id

Introduction

This report documents a relational database I designed and built for an e-commerce context. The scope was fairly wide — customers, orders, products, suppliers, payments, inventory — so the first real task was figuring out how to structure all of that without things getting tangled. I chose a relational model mainly because of the transactional guarantees it offers. ACID properties (Atomicity, Consistency, Isolation, Durability) aren't just theoretical niceties in a shopping system — they're what stops you ending up in a situation where a payment processes but the stock never decrements, or an order record is only half-written because something failed midway through.

I tried to stick to normalization principles throughout, partly because the module required it and partly because it genuinely makes the schema easier to maintain. To show the database working properly, I brought in joins, aggregation queries, indexes, stored procedures, a trigger, and manual transaction blocks.

The system isn't just transactional either. I wanted it to be useful for analysis — things like seeing which customers are spending the most, or which products are moving. That kind of reporting matters in a real e-commerce setup. I also made sure the constraints and transaction logic were embedded in the schema itself, not left to whatever application might sit on top of it.

System Design

The database has seven tables in total: `Customers`, `Orders`, `Order_Details`, `Products`, `Suppliers`, `Transactions`, and `Order_Log`. Each one connects to the others via primary and foreign keys — that's what keeps the referential integrity intact across the whole schema.

`Customers` stores the registration and contact information, and I put a `UNIQUE` constraint on the email column early on — without that, the same person could register twice without anything stopping them. The `Orders` table references `Customers` with `ON DELETE CASCADE`, which means if a customer gets deleted, their orders disappear too rather than leaving ghost records behind. `Order_Details` is a junction table sitting between `Orders` and `Products` to handle the many-to-many relationship. That's the bit that keeps the schema in Second Normal Form.

The `Products` table has `CHECK` constraints on price and stock — both have to be non-negative, which seems obvious but without the constraint you can't guarantee it at the data level. Supplier data lives in its own table rather than being repeated inside `Products`; that's what gets rid of the transitive dependency and brings things to 3NF. `Transactions` handles the payment side, linking each payment to an order, and `Order_Log` is there purely

for auditing — a trigger writes to it automatically when orders come in.

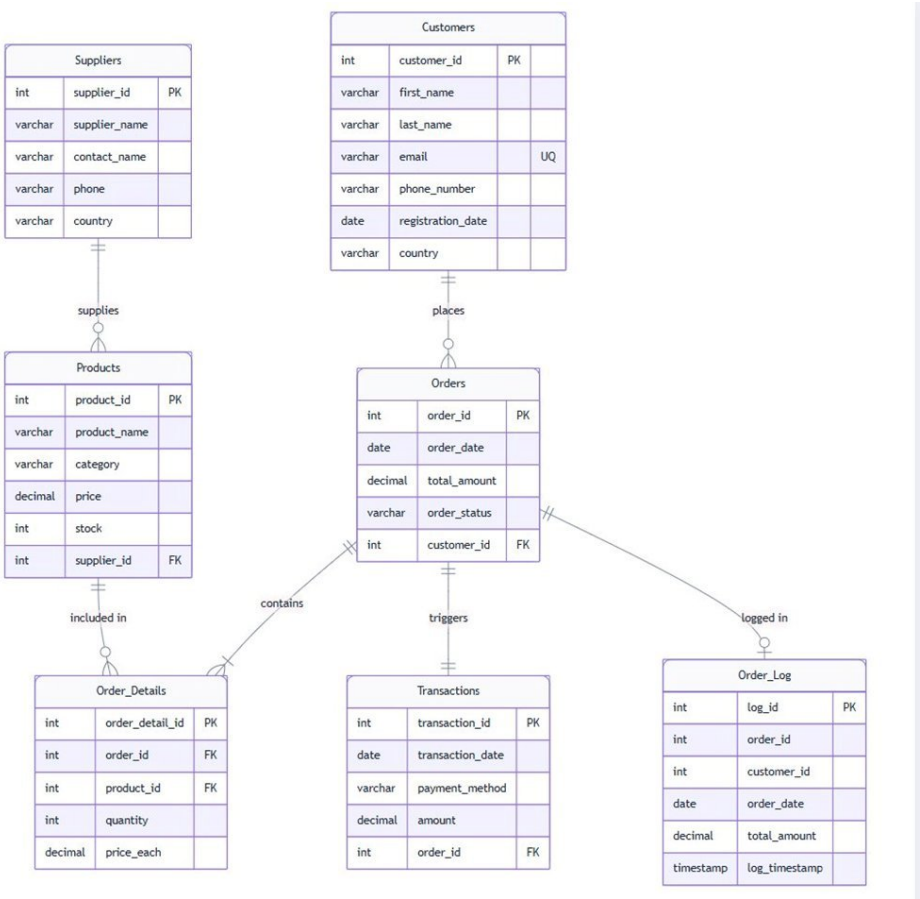


Figure 1: Entity-Relationship Diagram of the E-Commerce Database System

I started the design by thinking through what an e-commerce system actually does — customer accounts, buying products, suppliers providing stock, payments going through. Drawing out the ER diagram was useful here because it forced me to be precise about which things were genuinely separate entities and which relationships would become problems if I tried to flatten them.

Keeping `Products` and `Suppliers` separate was one of the earliest calls I made. Putting supplier data directly in the `Products` table would mean repeating it for every product that supplier provides — and then if the supplier changes their phone number, you're updating it in multiple rows rather than one. The `Order_Details` junction table came from the same kind of thinking. Orders and Products have a many-to-many relationship, and without a junction table you either repeat product data or break the normal form.

Translating the ER model into actual SQL tables meant going through each entity and working out the key constraints carefully. I did functional dependency checks on all of them to make sure no non-key column was depending on another non-key column — that's what 3NF requires. It took a bit of back-and-forth on a couple of tables but everything

came out right in the end.

Scale was something I had in the back of my mind even though the test data is small. The schema isn't going to need structural changes just because more records get added — the relationships are set up in a way that should hold up reasonably well as volumes increase.

Implementation

Everything was built in MySQL. `AUTO_INCREMENT` handles the primary keys, and I defined all the foreign key constraints explicitly rather than leaving them implied. `NOT NULL`, `UNIQUE`, and `CHECK` constraints are spread throughout — the idea being that bad data gets rejected at the schema level before it can cause issues further up.

Schema Creation

Below is the SQL schema used to create the seven tables:

```sql
CREATE TABLE Customers (
  customer_id        INT AUTO_INCREMENT PRIMARY KEY,
  first_name         VARCHAR(50)  NOT NULL,
  last_name          VARCHAR(50)  NOT NULL,
  email              VARCHAR(100) NOT NULL UNIQUE,
  phone_number       VARCHAR(20),
  registration_date  DATE         NOT NULL,
  country            VARCHAR(50)  NOT NULL
);

CREATE TABLE Suppliers (
  supplier_id   INT AUTO_INCREMENT PRIMARY KEY,
  supplier_name VARCHAR(100) NOT NULL,
  contact_name  VARCHAR(50),
  phone         VARCHAR(20),
  country       VARCHAR(50)
);

CREATE TABLE Products (
  product_id   INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(100)   NOT NULL,
  category     VARCHAR(50)    NOT NULL,
  price        DECIMAL(10,2)  NOT NULL CHECK (price > 0),
  stock        INT            NOT NULL DEFAULT 0 CHECK (stock
     >= 0),
  supplier_id  INT,
```

```sql
  FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

CREATE TABLE Orders (
  order_id     INT AUTO_INCREMENT PRIMARY KEY,
  order_date   DATE           NOT NULL,
  total_amount DECIMAL(10,2)  DEFAULT 0 CHECK (total_amount >=
     0),
  order_status VARCHAR(20)    DEFAULT 'Pending',
  customer_id  INT            NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
    ON DELETE CASCADE
);

CREATE TABLE Order_Details (
  order_detail_id INT AUTO_INCREMENT PRIMARY KEY,
  order_id        INT           NOT NULL,
  product_id      INT           NOT NULL,
  quantity        INT           NOT NULL CHECK (quantity > 0),
  price_each      DECIMAL(10,2) NOT NULL CHECK (price_each > 0)
     ,
  FOREIGN KEY (order_id)   REFERENCES Orders(order_id)   ON
     DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

CREATE TABLE Transactions (
  transaction_id   INT AUTO_INCREMENT PRIMARY KEY,
  transaction_date DATE          NOT NULL,
  payment_method   VARCHAR(50)   NOT NULL,
  amount           DECIMAL(10,2) NOT NULL CHECK (amount > 0),
  order_id         INT           NOT NULL,
  FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE
     CASCADE
);
```

Indexing Strategy

To improve query performance, I created indexes on the most frequently queried columns:

```sql
CREATE INDEX idx_customer_country    ON Customers(country);
CREATE INDEX idx_product_category    ON Products(category);
```

```
CREATE INDEX idx_order_date          ON Orders(order_date);
CREATE INDEX idx_order_customer      ON Orders(customer_id);
CREATE INDEX idx_orderdetails_product ON Order_Details(
   product_id);
```

Without indexes, every query that filters or joins on a column ends up scanning the whole table. Adding them to the most-used columns cuts that down significantly. Anchlia (2024) points to indexing as one of the more impactful things you can do for query responsiveness in relational systems, and from what I saw in testing, even with a small dataset the difference was noticeable on the join-heavy queries.

Normalization

The schema reaches all three normal forms. 1NF is straightforward — every column holds one value, so contact details are split into individual fields rather than being lumped into a single string. For 2NF, the `Order_Details` table is the important one: all the non-key columns there depend on the whole composite key, not just part of it. 3NF comes from pulling supplier data out of `Products` — leaving it there would mean a non-key column (supplier info) was functionally dependent on another non-key column (`supplier_id`), which is what 3NF is designed to prevent. Hamouda (2023) makes the argument that proper normalization is what makes schemas maintainable in the long run, and having gone through the process I think that's right — it's much easier to reason about tables when the dependencies are clean.

It also helps practically with updates. A supplier changing their contact details means one row to change in `Suppliers`, not tracking down every product row that references them.

Transactions and Business Logic

Partial updates are a real problem in transactional systems. If the stock decrement runs fine but the `order_details` insert fails for some reason, you've got inventory that's gone without an order to explain it. I used `START TRANSACTION` and `COMMIT` to wrap any operation that touches multiple tables, so either everything goes through together or `ROLLBACK` brings it back to where it was. Nothing gets left half-done.

Stored Procedure: PlaceOrder

`PlaceOrder` is a stored procedure that wraps the whole order placement sequence into one transactional block. It checks stock first, then inserts the order detail row, then updates the order total. I put it at the database layer on purpose — if that logic lived in the application instead, you'd have to trust every client to implement it correctly. At the database level it just runs the same way every time, regardless of what's calling it. Kraft

et al. (2023) make a similar point about transactional logic being essential for consistency in both relational and distributed systems, which reinforced that decision for me.

```
DELIMITER //
CREATE PROCEDURE PlaceOrder(
  IN p_order_id   INT,
  IN p_product_id INT,
  IN p_quantity   INT,
  IN p_price      DECIMAL(10,2)
)
BEGIN
  DECLARE available_stock INT;
  START TRANSACTION;

  SELECT stock INTO available_stock
  FROM Products WHERE product_id = p_product_id;

  IF available_stock >= p_quantity THEN
    UPDATE Products SET stock = stock - p_quantity
    WHERE product_id = p_product_id;

    INSERT INTO Order_Details(order_id, product_id, quantity,
       price_each)
    VALUES (p_order_id, p_product_id, p_quantity, p_price);

    UPDATE Orders
    SET total_amount = total_amount + (p_quantity * p_price)
    WHERE order_id = p_order_id;

    COMMIT;
  ELSE
    ROLLBACK;
  END IF;
END //
DELIMITER ;
```

Trigger: after_order_insert

The `after_order_insert` trigger fires on the `Orders` table every time a new row is inserted, and it writes the key details into `Order_Log` automatically. No manual step needed — the audit record just appears. That felt like a cleaner approach than relying on application code to remember to log every order.

```sql
CREATE TABLE Order_Log (
  log_id        INT AUTO_INCREMENT PRIMARY KEY,
  order_id      INT,
  customer_id   INT,
  order_date    DATE,
  total_amount  DECIMAL(10,2),
  log_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE TRIGGER after_order_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
  INSERT INTO Order_Log(order_id, customer_id, order_date,
     total_amount)
  VALUES (NEW.order_id, NEW.customer_id, NEW.order_date, NEW.
     total_amount);
END //
DELIMITER ;
```

There's a broader reason for putting rules like these in the database rather than the application. Application code can be bypassed — someone could run a direct SQL query against the database and skip whatever validation the app does. If the rule exists only at the database level, it applies no matter how the data is being written. That's a stronger guarantee.

Performance Optimization

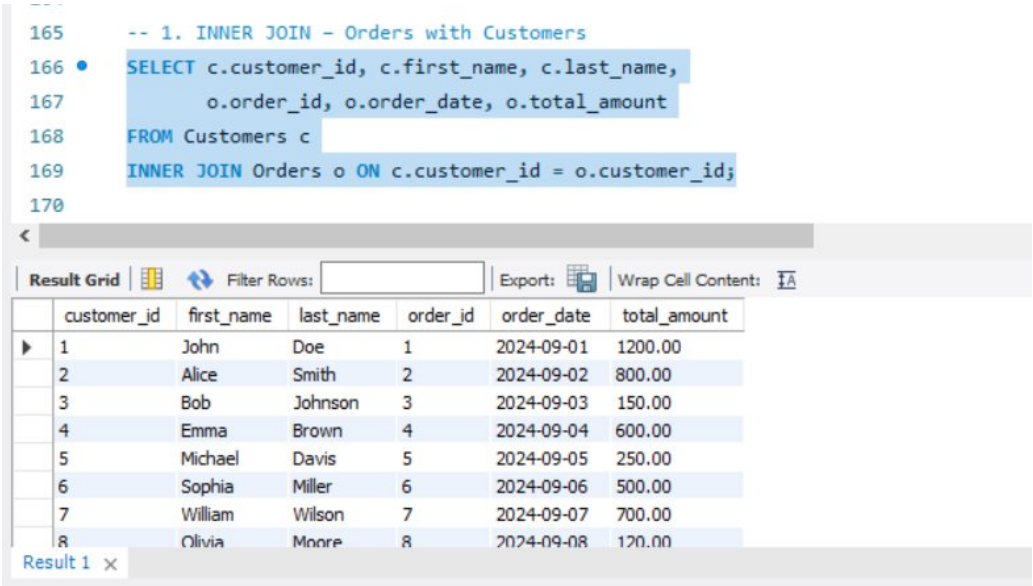
For the indexes, I focused on `customer_id`, `order_date`, `category`, and `country` — those are the columns that kept appearing in `WHERE` clauses and `JOIN` conditions when I was writing the queries. Testing with and without them showed a clear difference in how the queries ran, even with a relatively small dataset. The expectation is that the benefit becomes more pronounced as the data grows, so it made sense to set them up now rather than retrofitting later.

Results

Seven queries were run to test both the operational and analytical sides of the database. Screenshots of each result are included below.

Query 1: INNER JOIN — Orders with Customers

```sql
-- 1. INNER JOIN -- Orders with Customers
SELECT c.customer_id, c.first_name, c.last_name,
       o.order_id, o.order_date, o.total_amount
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id;
```

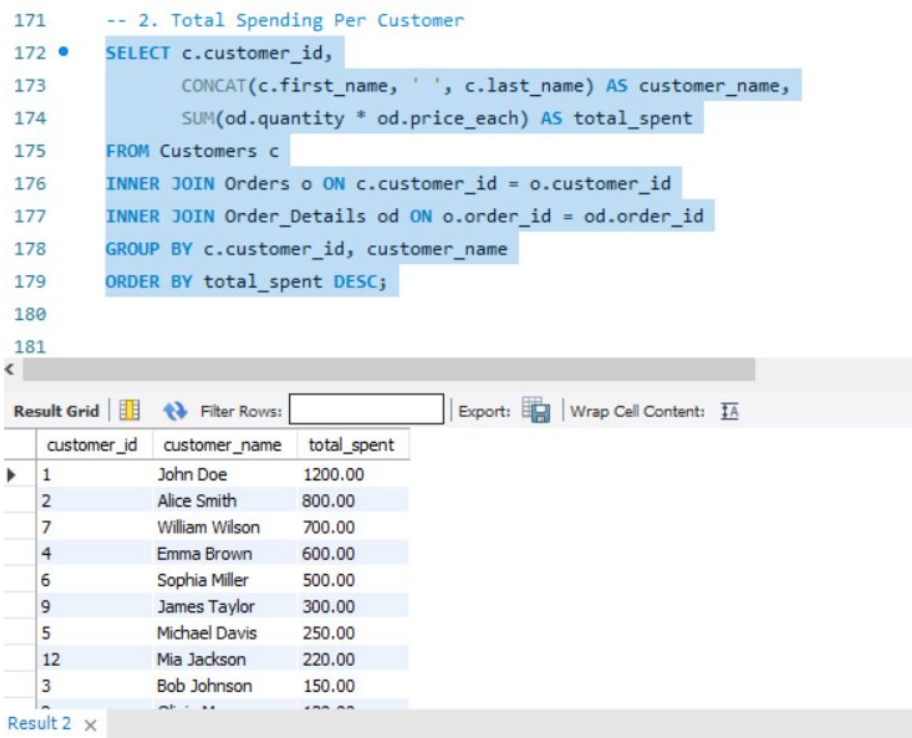


customer_id	first_name	last_name	order_id	order_date	total_amount
1	John	Doe	1	2024-09-01	1200.00
2	Alice	Smith	2	2024-09-02	800.00
3	Bob	Johnson	3	2024-09-03	150.00
4	Emma	Brown	4	2024-09-04	600.00
5	Michael	Davis	5	2024-09-05	250.00
6	Sophia	Miller	6	2024-09-06	500.00
7	William	Wilson	7	2024-09-07	700.00
8	Olivia	Moore	8	2024-09-08	120.00

Figure 2: INNER JOIN — Orders with Customers Result

Query 2: Total Spending Per Customer

```sql
-- 2. Total Spending Per Customer
SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       SUM(od.quantity * od.price_each)        AS total_spent
FROM Customers c
INNER JOIN Orders        o  ON c.customer_id = o.customer_id
INNER JOIN Order_Details od ON o.order_id    = od.order_id
GROUP BY c.customer_id, customer_name
ORDER BY total_spent DESC;
```



```
-- 2. Total Spending Per Customer
SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       SUM(od.quantity * od.price_each) AS total_spent
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id
INNER JOIN Order_Details od ON o.order_id = od.order_id
GROUP BY c.customer_id, customer_name
ORDER BY total_spent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

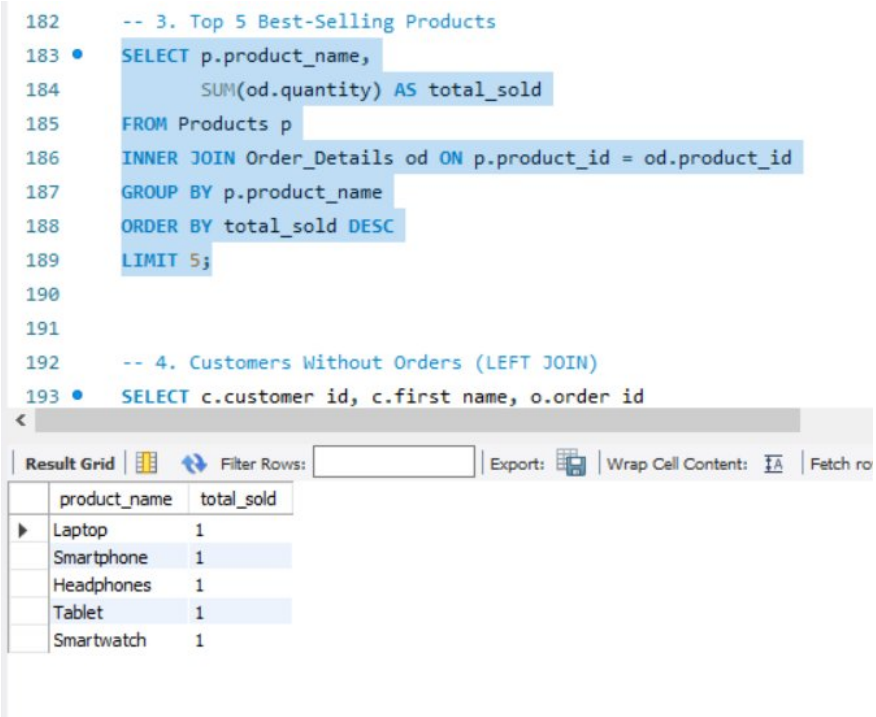
customer_id	customer_name	total_spent
1	John Doe	1200.00
2	Alice Smith	800.00
7	William Wilson	700.00
4	Emma Brown	600.00
6	Sophia Miller	500.00
9	James Taylor	300.00
5	Michael Davis	250.00
12	Mia Jackson	220.00
3	Bob Johnson	150.00

Result 2

Figure 3: Aggregation Query — Total Spending Per Customer

Query 3: Top 5 Best-Selling Products

```
-- 3. Top 5 Best-Selling Products
SELECT p.product_name,
       SUM(od.quantity) AS total_sold
FROM Products p
INNER JOIN Order_Details od ON p.product_id = od.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC
LIMIT 5;
```



```
-- 3. Top 5 Best-Selling Products
SELECT p.product_name,
       SUM(od.quantity) AS total_sold
FROM Products p
INNER JOIN Order_Details od ON p.product_id = od.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC
LIMIT 5;


-- 4. Customers Without Orders (LEFT JOIN)
SELECT c.customer_id, c.first_name, o.order_id
```

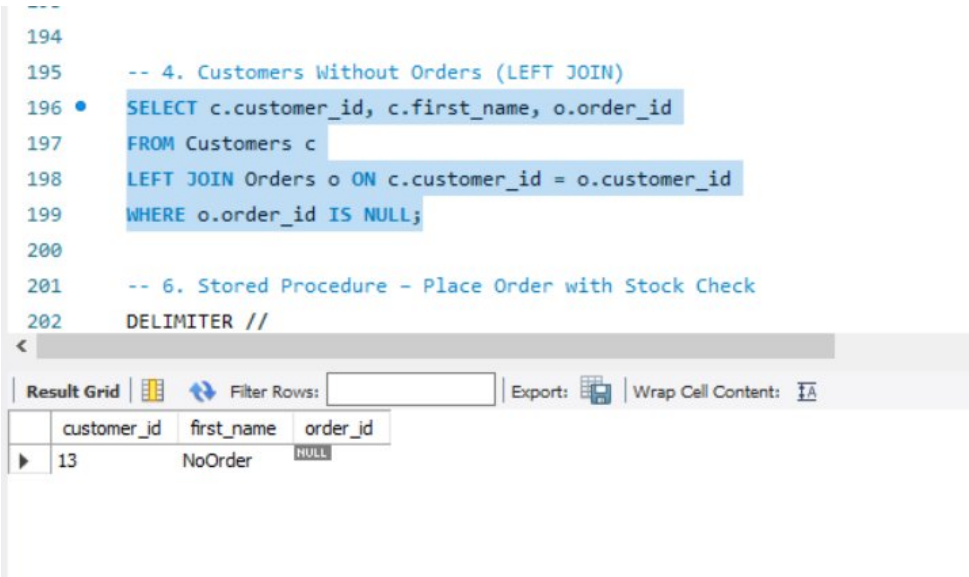
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch ro

product_name	total_sold
Laptop	1
Smartphone	1
Headphones	1
Tablet	1
Smartwatch	1

Figure 4: Aggregation Query — Top 5 Best-Selling Products

Query 4: Customers Without Orders (LEFT JOIN)

```
-- 4. Customers Without Orders (LEFT JOIN)
SELECT c.customer_id, c.first_name, o.order_id
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```



```
-- 4. Customers Without Orders (LEFT JOIN)
SELECT c.customer_id, c.first_name, o.order_id
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- 6. Stored Procedure - Place Order with Stock Check
DELIMITER //
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	order_id
13	NoOrder	NULL

Figure 5: LEFT JOIN — Customers Without Orders Result

Query 5: Revenue by Product Category

```sql
-- 5. Revenue by Product Category
SELECT p.category,
       COUNT(od.order_detail_id) AS total_orders,
       SUM(od.quantity * od.price_each) AS revenue
FROM Products p
INNER JOIN Order_Details od ON p.product_id = od.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

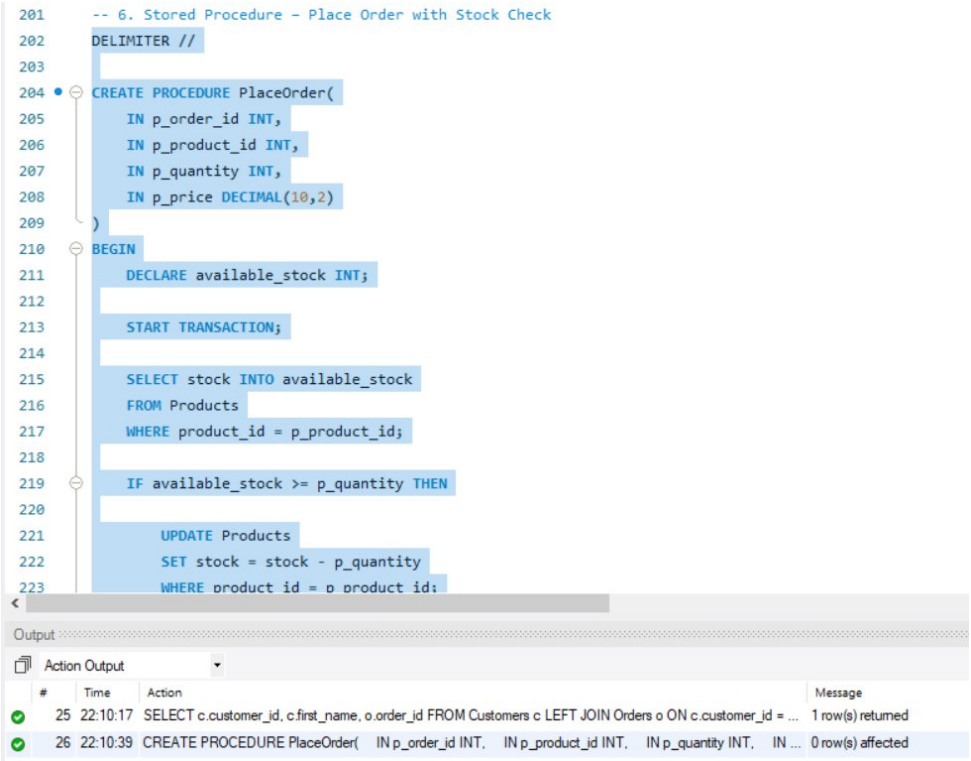


Figure 6: Aggregation Query — Revenue by Product Category

Query 6: Orders by Status Count

```sql
-- 6. Orders by Status Count
SELECT order_status,
       COUNT(*) AS total_orders,
       SUM(total_amount) AS total_value
FROM Orders
GROUP BY order_status
ORDER BY total_orders DESC;
```

Figure 7: Aggregation Query — Orders by Status Count

Query 7: Stored Procedure, Trigger & Manual Transaction

Stored Procedure — PlaceOrder with Stock Check:

```
DROP PROCEDURE IF EXISTS PlaceOrder;
DELIMITER //
CREATE PROCEDURE PlaceOrder(
  IN p_order_id   INT,
  IN p_product_id INT,
  IN p_quantity   INT,
  IN p_price      DECIMAL(10,2)
)
BEGIN
  DECLARE available_stock INT DEFAULT 0;
  DECLARE order_exists    INT DEFAULT 0;

  SELECT COUNT(*) INTO order_exists
  FROM Orders WHERE order_id = p_order_id;

  SELECT stock INTO available_stock
  FROM Products WHERE product_id = p_product_id;
```

```
  SELECT available_stock AS current_stock,
         p_quantity      AS requested_qty,
         order_exists    AS order_found;

  IF order_exists = 0 THEN
    SELECT 'FAILED: order_id does not exist' AS result;

  ELSEIF available_stock >= p_quantity THEN
    START TRANSACTION;

    UPDATE Products SET stock = stock - p_quantity
    WHERE product_id = p_product_id;

    INSERT INTO Order_Details(order_id, product_id, quantity,
       price_each)
    VALUES(p_order_id, p_product_id, p_quantity, p_price);

    UPDATE Orders
    SET total_amount = total_amount + (p_quantity * p_price)
    WHERE order_id = p_order_id;

    COMMIT;
    SELECT 'SUCCESS: Order placed.' AS result;

  ELSE
    SELECT CONCAT('FAILED: Insufficient stock. Available: ',
                  available_stock) AS result;
  END IF;
END //
DELIMITER ;

CALL PlaceOrder(5, 17, 3, 800.00);
SELECT product_id, stock FROM Products WHERE product_id = 17;
```

Trigger — Auto-log New Orders:

```
CREATE TABLE IF NOT EXISTS Order_Log (
  log_id        INT AUTO_INCREMENT PRIMARY KEY,
  order_id      INT,
  customer_id   INT,
```

```sql
  order_date    DATE,
  total_amount  DECIMAL(10,2),
  log_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DROP TRIGGER IF EXISTS after_order_insert;
DELIMITER //
CREATE TRIGGER after_order_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
  INSERT INTO Order_Log(order_id, customer_id, order_date,
     total_amount)
  VALUES (NEW.order_id, NEW.customer_id, NEW.order_date, NEW.
     total_amount);
END //
DELIMITER ;

INSERT INTO Orders(order_date, total_amount, order_status,
   customer_id)
VALUES ('2024-10-01', 450.00, 'Pending', 1);

SELECT * FROM Order_Log ORDER BY log_timestamp DESC LIMIT 3;
```

Manual Transaction Example:

```sql
START TRANSACTION;

UPDATE Products SET stock = stock - 2 WHERE product_id = 1;

INSERT INTO Order_Details(order_id, product_id, quantity,
   price_each)
VALUES (1, 1, 2, 1200.00);

COMMIT;
```

```
-- 8. Manual Transaction Example
START TRANSACTION;

UPDATE Products
SET stock = stock - 2
WHERE product_id = 1;

INSERT INTO Order_Details(order_id, product_id, quantity, price_each)
VALUES (1,1,2,1200.00);

COMMIT;
```

Output

Action Output

#	Time	Action
31	22:11:25	INSERT INTO Order_Details(order_id, product_id, quantity, price_each) VALUES (1,1,2,1200.00)
32	22:11:25	COMMIT

Figure 8: Stored Procedure, Trigger & Manual Transaction Results

All seven queries ran without errors. The `INNER JOIN` brought back the customer-order pairs correctly. The aggregation query sorted customers by total spend in descending order, which worked as expected. The `LEFT JOIN` identified customers with no orders — useful if you wanted to follow up with them for marketing. The stored procedure and trigger both fired correctly too, which was the main thing I wanted to confirm, since those involved the most moving parts.

Critical Evaluation and System Justification

Stepping back from the implementation, I think there's a reasonable gap between what I've built here and what a production system would actually need. The core design works, but it's worth being clear-eyed about what's missing.

The normalization is probably the strongest part. Getting to 3NF meant being quite disciplined about where data lives, and the result is a schema with very little redundancy and no obvious update anomalies. Putting validation logic into stored procedures and triggers rather than relying on the application is something I'd stand behind too — it means the rules are enforced at the data layer, not just in one particular client.

The analytical queries also came out well. Ranking customers by spend, finding the best-selling products, spotting unengaged customers — these are questions a real business would actually ask, and the schema answers them without needing any structural changes. That

felt like a good sign that the design reflects something close to real-world requirements.

Where it falls short is scale. Indexing helps with query speed, but at high data volumes you'd probably need table partitioning or a move toward distributed SQL. Dong et al. (2024) and Pina et al. (2022) both cover how relational systems are evolving to handle horizontal scaling — things like CockroachDB and VoltDB — and this system isn't anywhere near that. For a prototype or small deployment it's fine; for something handling millions of orders, probably not.

There's also a genuine tension in strict normalization that I noticed when writing the more complex queries. 3NF keeps the data clean, but it means more joins — and joins get expensive at scale. In a read-heavy system you might actually want to denormalize some things deliberately, accepting a bit of redundancy in exchange for faster reads. It's one of those cases where the theoretically correct answer isn't always the most practical one.

That said, for what it is, the system works. Transactional integrity is solid, the analytical queries do something genuinely useful, and embedding business logic in the database rather than the application gives it more consistency than it would otherwise have.

Conclusion and Future Work

This project was more involved than I expected, particularly the normalization side of things. Getting to 3NF isn't just a checklist — it requires actually thinking through the dependencies in each table and restructuring things when they're not right. The implementation side — transactions, indexes, stored procedures, triggers — came together reasonably well, and I think the different parts complement each other in a way that makes the overall system fairly robust for its scope.

If I were extending this, the first things I'd tackle are security and reporting. There's no access control at all right now, and no password hashing — both of which would be non-negotiable in anything handling real customer data. Reporting views would also be worth adding to simplify the more complex analytical queries. Further down the line, composite indexes and partitioning would help if transaction volumes grew significantly.

What I took away from the process most was how much design decisions early on affect what's possible later. Getting the normalization right meant the queries could be written cleanly. Having the foreign keys set up properly meant cascade deletes worked without extra code. The trigger and stored procedure only make sense because the underlying schema is structured well enough to support them. It's the kind of thing that's hard to appreciate until you've actually had to work through it.

References

Anchlia, A., 2024. Enhancing query performance through relational database indexing. *International Journal of Computer Trends and Technology*, 72(8), pp.130–133. Available at: https://www.ijcttjournal.org/Volume-72%20Issue-8/IJCTT-V72I8P119.pdf

Dong, H., Chen, Y., Wang, J., et al., 2024. Cloud-native databases: A survey. *IEEE Transactions on Knowledge and Data Engineering*. Available at: https://ieeexplore.ieee.org/document/10411783

Hamouda, S., 2023. Normalization and denormalization for a document-oriented database. In: *Proceedings of the 25th International Conference on Enterprise Information Systems (ICEIS 2023)*, pp.451–458. Available at: https://www.scitepress.org/Papers/2023/118681/118681.pdf

Kraft, P., Li, Q., Zhou, X., Bailis, P., Stonebraker, M., Yu, X. and Zaharia, M., 2023. Epoxy: ACID transactions across diverse data stores. *Proceedings of the VLDB Endowment*, 16(11), pp.2742–2754. DOI: https://doi.org/10.14778/3611479.3611484

Pina, E., Leitão, J., Vilaça, R. and Rodrigues, L., 2022. NewSQL databases assessment: CockroachDB, MariaDB Xpand, and VoltDB. *Future Internet*, 15(1), p.10. DOI: https://doi.org/10.3390/fi15010010